K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

SCHOOL OF ENGINEERING AND TECHNOLOGY

Basics of Neural Network & Deep Learning Lab ENCA354

BCA (AI & DS)

LAB FILE

Submitted By:
Student Name: Disha Saini
Roll No: 2301201114
Programme Name: BCA (AI & DS)
Semester: 6th

Submitted to:
Mr. Anoja Mishra

INDEX

SR. NO.	EXPERIMENTS	PAGE NO.	TEACHER SIGN
01	Neural Network using TensorFlow (Low-Level API)	**3**	
02	Neural Network using Basic Python (From Scratch)	**5**	
03	Neural Network using Keras	**7**	
04	Agriculture Price Prediction (Regression Problem)	**9**	
05	MNIST Digit Classification using ANN	**11**	
06	Plant Disease Detection using CNN	**13**	

Projects: Basics of Neural Network and Deep Learning

1. Neural Network using TensorFlow (Low-Level API)

- **Build a simple feedforward neural network using TensorFlow 1.x style (placeholders, sessions)**
- **Understand computation graph and manual weight initialization**

ANS:

Plant disease detection is performed using a Convolutional Neural Network (CNN) to classify leaf images. A subset of the PlantVillage dataset is used for training. The model extracts features using convolution and pooling layers. These features are passed to dense layers for classification. The model is trained using Adam optimizer and categorical crossentropy loss. It achieves good accuracy and is saved for future predictions.

```
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
import numpy as np

# ---------------- SIMPLE DATA (XOR) ----------------
x_data = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=np.float32)
y_data = np.array([[0],[1],[1],[0]], dtype=np.float32)

# ---------------- INPUT PLACEHOLDERS ----------------
x = tf.placeholder(tf.float32, shape=[None, 2])  # input
y = tf.placeholder(tf.float32, shape=[None, 1])  # output

# ---------------- WEIGHTS & BIASES ----------------
# hidden layer (2 inputs -> 3 neurons)
w1 = tf.Variable(tf.random_normal([2, 3]))
b1 = tf.Variable(tf.zeros([3]))

# output layer (3 -> 1)
w2 = tf.Variable(tf.random_normal([3, 1]))
b2 = tf.Variable(tf.zeros([1]))

# ---------------- FORWARD PASS ----------------
hidden = tf.nn.relu(tf.matmul(x, w1) + b1)   # hidden layer
output = tf.sigmoid(tf.matmul(hidden, w2) + b2)  # final output

# ---------------- LOSS FUNCTION ----------------
loss = tf.reduce_mean(tf.square(output - y))
```

```python
# ---------------- TRAINING ----------------
train = tf.train.GradientDescentOptimizer(0.1).minimize(loss)

# ---------------- RUN SESSION ----------------
with tf.Session() as sess:
    sess.run(tf.global_variables_initializer())

    for i in range(3000):
        sess.run(train, feed_dict={x: x_data, y: y_data})

        if i % 500 == 0:
            l = sess.run(loss, feed_dict={x: x_data, y: y_data})
            print("Step:", i, "Loss:", l)

    # ---------------- RESULT ----------------
    result = sess.run(output, feed_dict={x: x_data})
    print("\nPredictions:\n", result)
```

```
WARNING:tensorflow:From /usr/local/lib/python3.12/dist-packages/tensorflow/python/comp
Instructions for updating:
non-resource variables are not supported in the long term
Step: 0 Loss: 0.2805721
Step: 500 Loss: 0.14938907
Step: 1000 Loss: 0.13011928
Step: 1500 Loss: 0.12712069
Step: 2000 Loss: 0.12623613
Step: 2500 Loss: 0.12585638

Predictions:
 [[0.50122374]
```

```
Predictions:
 [[0.50122374]
 [0.9611479 ]
 [0.50122374]
 [0.03006977]]
```

2.Neural Network using Basic Python (From Scratch)

- **Implement a neural network using NumPy only**
- **Include:**
  - **Forward propagation**
  - **Backpropagation**
  - **Gradient descent**

ANS:

A neural network is implemented using NumPy to understand its internal working. In forward propagation, input data passes through layers using weights and activation functions to produce output. In backpropagation, the error between predicted and actual output is calculated and propagated backward. Gradients are computed to update weights. Gradient descent is used to minimize the loss by adjusting weights. This process repeats until the model learns correct predictions.

```python
import numpy as np

# ---------------- DATA (XOR) ----------------
X = np.array([[0,0],[0,1],[1,0],[1,1]])   # input
Y = np.array([[0],[1],[1],[0]])           # output

# ---------------- ACTIVATION FUNCTIONS ----------------
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# ---------------- INITIALIZE WEIGHTS ----------------
np.random.seed(1)

W1 = np.random.randn(2, 3)   # input → hidden
b1 = np.zeros((1, 3))

W2 = np.random.randn(3, 1)   # hidden → output
b2 = np.zeros((1, 1))

learning_rate = 0.1

# ---------------- TRAINING ----------------
for epoch in range(5000):
```

```python
# -------- FORWARD PROPAGATION --------
Z1 = np.dot(X, W1) + b1
A1 = sigmoid(Z1)

Z2 = np.dot(A1, W2) + b2
output = sigmoid(Z2)

# -------- LOSS (MSE) --------
loss = np.mean((Y - output) ** 2)

# -------- BACKPROPAGATION --------
d_output = (output - Y) * sigmoid_derivative(output)

d_W2 = np.dot(A1.T, d_output)
d_b2 = np.sum(d_output, axis=0, keepdims=True)

d_hidden = np.dot(d_output, W2.T) * sigmoid_derivative(A1)

d_W1 = np.dot(X.T, d_hidden)
d_b1 = np.sum(d_hidden, axis=0, keepdims=True)

# -------- GRADIENT DESCENT --------
W2 -= learning_rate * d_W2
b2 -= learning_rate * d_b2

W1 -= learning_rate * d_W1
b1 -= learning_rate * d_b1

# -------- PRINT LOSS --------
```

```python
    # -------- PRINT LOSS --------
    if epoch % 500 == 0:
        print("Epoch:", epoch, "Loss:", loss)

# ---------------- FINAL OUTPUT ----------------
print("\nFinal Predictions:\n", output)
```

```
Epoch: 0 Loss: 0.29710433386836943
Epoch: 500 Loss: 0.23211128977121462
Epoch: 1000 Loss: 0.1961332191194099
Epoch: 1500 Loss: 0.16469902265406508
Epoch: 2000 Loss: 0.1478346837589441
Epoch: 2500 Loss: 0.12399239885490537
Epoch: 3000 Loss: 0.06662401389421313
Epoch: 3500 Loss: 0.03304289300469073
Epoch: 4000 Loss: 0.019624680838915748
Epoch: 4500 Loss: 0.013308227279985006

Final Predictions:
 [[0.05970438]
 [0.89530496]
 [0.91729751]
 [0.13409994]]
```

3. Neural Network using Keras

- **Build a model using Keras Sequent l API**
  - **Train on a small dataset (e.g., Iris or dummy data)**
  - **Visualize accuracy and loss**

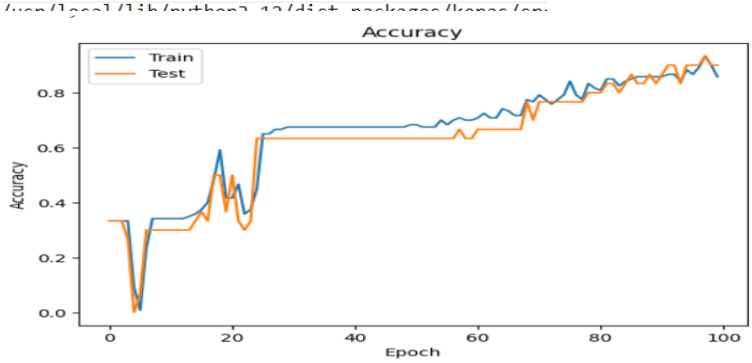
ANS:

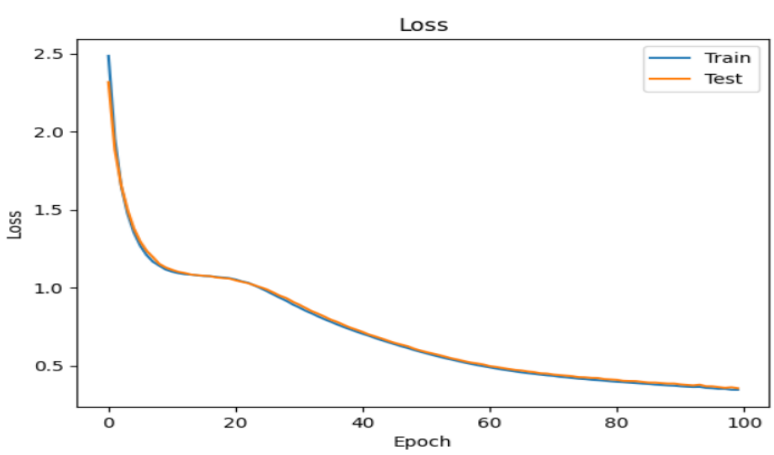
MNIST digit classification is a supervised learning task where handwritten digit images are classified into digits (0–9). The MNIST dataset consists of 28×28 grayscale images. These images are flattened and given as input to an Artificial Neural Network (ANN). The model has fully connected layers that learn patterns in the data. It is trained using an optimizer and loss function to improve accuracy. The model performance is evaluated using accuracy and confusion matrix.

```python
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load dataset
data = load_iris()
X = data.data
y = data.target.reshape(-1, 1)

# Convert output to one-hot
encoder = OneHotEncoder(sparse_output=False) # Changed 'sparse' to 'sparse_output'
y = encoder.fit_transform(y)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Build model
model = Sequential()
model.add(Dense(8, input_dim=4, activation='relu'))
model.add(Dense(6, activation='relu'))
model.add(Dense(3, activation='softmax'))

# Compile model
```

```python
# Compile model
model.compile(loss='categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

# Train model
history = model.fit(X_train, y_train,
                    epochs=100,
                    batch_size=5,
                    validation_data=(X_test, y_test),
                    verbose=0)

# Evaluate
loss, acc = model.evaluate(X_test, y_test)
print("Accuracy:", acc)

# Plot Accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Test"])
plt.show()

# Plot Loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Loss")
```

```python
# Plot Loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Test"])
plt.show()
```





4. Agriculture Price Prediction (Regression Problem)

- **Take any dataset from GitHub (e.g., crop price dataset)**
- **Build a regression model using ANN**
- **Evaluate using:**
  - **MSE**
  - **R^2 Score**

ANS:

A neural network can be implemented using TensorFlow 1.x low-level API to understand its internal working. In this approach, placeholders are used to provide input data, and variables are used to define weights and biases. A computation graph is created where operations like matrix multiplication and activation functions are defined. The graph is executed using a session. The model is trained using an optimizer to minimize the loss function. This method gives better understanding of how neural networks work internally.

```python
import pandas as pd

df = pd.read_csv("/content/crop_price_dataset.csv")
print("File loaded successfully into a DataFrame. First 5 rows:")
print(df.head())
```

```
File loaded successfully into a DataFrame. First 5 rows:
        month commodity_name  avg_modal_price  avg_min_price  avg_max_price  \
0  2025-03-01          Maize          2341.58        2191.23        2402.98
1  2025-02-01          Maize          2356.01        2202.35        2414.46
2  2025-01-01          Maize          2382.20        2221.31        2448.22
3  2024-12-01          Maize          2328.78        2164.86        2411.66
4  2024-11-01          Maize          2273.46        2077.63        2377.78

  state_name district_name calculationType  change
0      India           All         Monthly  -14.43
1      India           All         Monthly  -26.19
2      India           All         Monthly   53.42
3      India           All         Monthly   55.32
4      India           All         Monthly   35.48
```

```python
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# ---------------- LOAD DATA ----------------
# Replace filename with your uploaded file name
df = pd.read_csv("/content/crop_price_dataset.csv")

print(df.head())   # see dataset

# ---------------- SELECT FEATURES ----------------
# CHANGE these column names according to your dataset

X = df[['avg_min_price', 'avg_max_price', 'change']]   # input features
y = df['avg_modal_price']                              # target

# Handle missing values by dropping rows with NaNs in X or y
data = pd.concat([X, y], axis=1).dropna()
X = data[['avg_min_price', 'avg_max_price', 'change']]
y = data['avg_modal_price']

# ---------------- TRAIN TEST SPLIT ----------------
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```python
# ---------------- SCALING ----------------
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ---------------- BUILD MODEL ----------------
model = Sequential()
model.add(Dense(8, inp Loading... rain.shape[1], activation='relu'))
model.add(Dense(4, activation='relu'))
model.add(Dense(1))

# ---------------- COMPILE ----------------
model.compile(optimizer='adam', loss='mse')

# ---------------- TRAIN ----------------
model.fit(X_train, y_train, epochs=100, verbose=0)

# ---------------- PREDICT ----------------
y_pred = model.predict(X_test)

# ---------------- EVALUATE ----------------
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MSE:", mse)
print("R2 Score:", r2)
```

```
...          month commodity_name  avg_modal_price  avg_min_price  avg_max_price  \
0  2025-03-01           Maize          2341.58        2191.23        2402.98
1  2025-02-01           Maize          2356.01        2202.35        2414.46
2  2025-01-01           Maize          2382.20        2221.31        2448.22
3  2024-12-01           Maize          2328.78        2164.86        2411.66
4  2024-11-01           Maize          2273.46        2077.63        2377.78

  state_name district_name calculationType  change
0      India           All          Monthly  -14.43
1      India           All          Monthly  -26.19
2      India           All          Monthly   53.42
3      India           All          Monthly   55.32
4      India           All          Monthly   35.48
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do n
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
MSE: 842412.6912245702
R2 Score: 0.9657204064038128
```

5. MNIST Digit Classification using ANN

- **Load MNIST dataset**
- **Build a simple ANN (fully connected network)**
- **Evaluate accuracy and confusion matrix**

ANS:

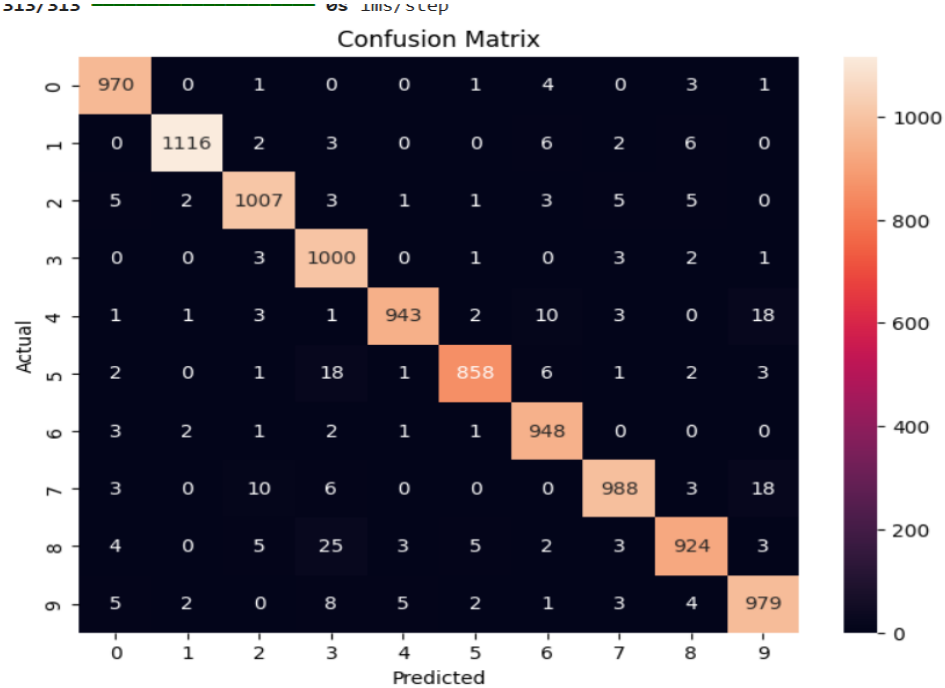
Keras is a high-level deep learning API used to build neural networks easily. In this project, the Sequential API is used to create a model with multiple dense layers. The model is trained on the Iris dataset for classification. Keras automatically handles forward and backpropagation, making implementation simple. The model is compiled using an optimizer and loss function, and trained for multiple epochs. The performance is evaluated using accuracy and loss graphs.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import seaborn as sns

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# ---------------- LOAD DATA ----------------
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# ---------------- PREPROCESS ----------------
# Flatten 28x28 → 784
X_train = X_train.reshape(-1, 784) / 255.0
X_test = X_test.reshape(-1, 784) / 255.0

# ---------------- BUILD MODEL ----------------
model = Sequential()
model.add(Dense(128, input_dim=784, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))

# ---------------- COMPILE ----------------
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```python
# ---------------- TRAIN ----------------
model.fit(X_train, y_train, epochs=5, batch_size=32, verbose=1)

# ---------------- EVALUATE ----------------
loss, accuracy = model.evaluate(X_test, y_test)
print("\nAccuracy:", accuracy)

# ---------------- CONFUSION MATRIX ----------------
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)

cm = confusion_matrix(y_test, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.9292 - loss: 0.2378
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9694 - loss: 0.1010
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9778 - loss: 0.0706
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9831 - loss: 0.0534
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9866 - loss: 0.0416
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9733 - loss: 0.0878

Accuracy: 0.9732999801635742
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step



6. Plant Disease Detection using CNN

- **Use plant leaf dataset (e.g., from Kaggle)**
- **Multi-class classification of diseases**
- **Save basic prediction model**

ANS:

Plant disease detection is performed using a Convolutional Neural Network (CNN) to classify leaf images into different disease categories. A subset of the PlantVillage dataset is used for training. CNN extracts features like edges and patterns using convolution and pooling layers. These features are passed to fully connected layers for classification. The model is trained using an optimizer and loss function to achieve good accuracy. The trained model is saved for future prediction.

```
!wget https://github.com/spMohanty/PlantVillage-Dataset/archive/refs/heads/master.zip
!unzip master.zip
```

```
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bcbe4121-e763-411e-bb1a-f438266427c4___UF.GRC_YLCV_Lab 02511_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bcd05e57-4a54-41f0-a586-18abba3c1caa___UF.GRC_YLCV_Lab 03035_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bcd5e29a-60ab-4885-bdda-d56ae1cdfd65___YLCV_NREC 2226_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bd0da569-2f56-43eb-9d27-406343f5237e___YLCV_GCREC 2084_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bd0e0dde-52fd-4c18-ad4e-0e3cc10c920f___UF.GRC_YLCV_Lab 08441_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bd16a603-6505-4189-89c2-7341b1a695ac___YLCV_GCREC 5326_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bd1bd3ed-6a34-417e-aa88-e587615174df___UF.GRC_YLCV_Lab 01449_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bd24a639-b272-4104-bc84-b9cabc628893___YLCV_GCREC 5462_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bd257fb0-a603-46e4-9c36-23963eded620___YLCV_NREC 2252_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bd596379-8494-4a58-92fb-4403589b9863___YLCV_GCREC 2286_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bd7593bc-0d9e-4579-9648-685d94e8bd44___UF.GRC_YLCV_Lab 03354_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bd82a16d-373d-4436-b5cb-efd84c4faf2a___UF.GRC_YLCV_Lab 02678_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bd8dade5-7c82-40ad-b12f-9dcc1e394e76___UF.GRC_YLCV_Lab 03342_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bda5e804-a531-4395-9043-647a573a0d43___YLCV_GCREC 2493_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bdb177f1-13d0-4a45-b197-aa20ef75b4ca___UF.GRC_YLCV_Lab 02750_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bdb46bf4-d226-4cd1-be9d-4ca36623fd5f___UF.GRC_YLCV_Lab 03274_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bdba2e33-7a1d-4886-a654-bdc78fe9cacd___UF.GRC_YLCV_Lab 02201_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bdba6b3a-019c-432d-921f-d5cf7614687b___UF.GRC_YLCV_Lab 02575_final_masked.jpg
inflating: PlantVillage-Dataset-master/raw/segmented/Tomato___Tomato_Yellow_Leaf_Curl_Virus/bdba7a1e-06b5-4617-a43c-2c723a485319___YLCV_NREC 2068_final_masked.jpg
```

```
import os
import shutil

source = "/content/PlantVillage-Dataset-master/raw/color"
destination = "/content/small_dataset"

# choose only 3 classes
classes = os.listdir(source)[:3]

os.makedirs(destination, exist_ok=True)

for cls in classes:
    shutil.copytree(os.path.join(source, cls),
                    os.path.join(destination, cls))

print("Small dataset ready!")
print(os.listdir(destination))
```

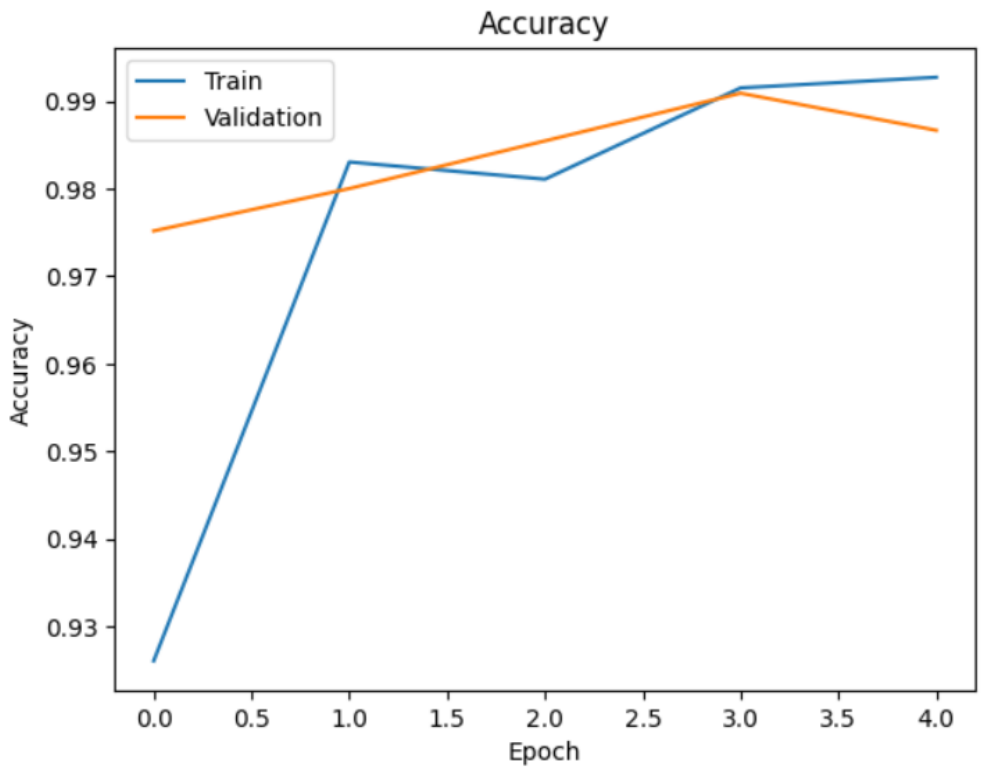
```
Small dataset ready!
['Orange___Haunglongbing_(Citrus_greening)', 'Apple___Apple_scab', 'Tomato___Bacterial_spot']
```

```python
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
import matplotlib.pyplot as plt

# ---------------- DATA PATH ----------------
data_path = "/content/small_dataset"

# ---------------- IMAGE PREPROCESSING ----------------
datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_data = datagen.flow_from_directory(
    data_path,
    target_size=(128,128),
    batch_size=32,
    class_mode='categorical',
    subset='training'
)

val_data = datagen.flow_from_directory(
    data_path,
    target_size=(128,128),
    batch_size=32,
    class_mode='categorical',
    subset='validation'
)
```

```python
# ---------------- CNN MODEL ----------------
model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)))
model.add(MaxPooling2D(2,2))

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))

model.add(Flatten())

model.add(Dense(128, activation='relu'))
model.add(Dense(train_data.num_classes, activation='softmax'))

# ---------------- COMPILE ----------------
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# ---------------- TRAIN ----------------
history = model.fit(train_data,
                    validation_data=val_data,
                    epochs=5)

# ---------------- SAVE MODEL ----------------
model.save("plant_disease_model.h5")
print("Model saved successfully!")
```

```python
# ---------------- PLOT ----------------
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train","Validation"])
plt.show()
```

```
Found 6612 images belonging to 3 classes.
Found 1652 images belonging to 3 classes.
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
207/207 ━━━━━━━━━━━━━━━━━━━━ 179s 852ms/step - accuracy: 0.9260 - loss: 0.2067 - val_accuracy: 0.9752 - val_loss: 0.0689
Epoch 2/5
207/207 ━━━━━━━━━━━━━━━━━━━━ 171s 828ms/step - accuracy: 0.9831 - loss: 0.0495 - val_accuracy: 0.9800 - val_loss: 0.0670
Epoch 3/5
207/207 ━━━━━━━━━━━━━━━━━━━━ 164s 791ms/step - accuracy: 0.9811 - loss: 0.0508 - val_accuracy: 0.9855 - val_loss: 0.0499
Epoch 4/5
207/207 ━━━━━━━━━━━━━━━━━━━━ 162s 785ms/step - accuracy: 0.9915 - loss: 0.0227 - val_accuracy: 0.9909 - val_loss: 0.0393
Epoch 5/5
207/207 ━━━━━━━━━━━━━━━━━━━━ 171s 825ms/step - accuracy: 0.9927 - loss: 0.0195 - val_accuracy: 0.9867 - val_loss: 0.0334
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using inste
Model saved successfully!
```


Accuracy
Train
Validation
0.99
0.98
0.97
0.96
0.95
0.94
0.93
Accuracy
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch